IMPORTANT DIFFERENCE between LOG files from [mcr_2405] –vs– [2603] versions of DMON firmware/d3 programs:

- **[2603]** uses **microseconds** represented by **8-character** field for $DUTY lines, but milliseconds (4-character field) for $START/$END lines
- [mcr_2405] uses millisecond 4-character field; **1 millisecond = 1000 microseconds**

UVI2_PRSTT_North_20260428 mission used **[2603]** version DMON update release
*(firmware **mcrec2603.bin**, d3host.exe **version 2.4**, d3read.exe **version 2.1**, and reformat_dmon_wav_files.sav **version 2**)*

The data payload and checksum of $DUTY lines in DMON2 log files can be decoded as follows…

Example $DUTY line in log file (from UVI2_PRSTT_North_20260428_050.log):
1778722792,53,$DUTY,6A0527A9000F3A836A0527E5000F3A7F30000104A00DBBA00*00

Characters 2-9 (n=8) after $DUTY: Start date-time of recording session in Unix epoch UTC
6A0527A9 hexadecimal = **1778722729** decimal (sec Unix epoch) = **2026-05-14 01:38:49** UTC

Characters 10-17 (n=8): Microseconds after start date-time in Unix epoch UTC
000F3A83 hex = **998019** decimal (microseconds) = **0.998019** seconds →
2026-05-14 01:38:49 UTC + 998019 microseconds = **2026-05-14 01:38:49.998019** UTC

Characters 18-25 (n=8): End date-time of recording session in Unix epoch UTC
6A0527E5 hex = **1778722789** decimal (sec Unix epoch) = **2026-05-14 01:39:49** UTC

Characters 26-33 (n=8): Microseconds after end dt in Unix epoch UTC
000F3A7F hex = **998015** decimal (microseconds) = **0.998015** seconds →
2026-05-14 01:39:49 UTC + 998015 microseconds = **2026-05-14 01:39:49.998015** UTC

Character 34 (n=1): Hydrophone numeric ID **2 = LF hydrophone, 3 = HF hydrophone, 1 = MF hydrophone*
3 = HF hydrophone

Characters 35-42 (n=8): Sequential recording session number for *specific hydrophone (LF/HF)* during mission
0000104A hex = **4170** decimal = **4,170th** recording session for HF hydrophone during mission

Characters 43-50 (n=8): Number of samples in current recording session
00DBBA00 hex = **14400000** decimal = **14,400,000** total samples in the 4,170th HF recording session

Characters 52-53 (n=2): Checksum indicating if any errors exist in the line
00 **see last page of this tab for details on calculating the checksum for each line*

OPTIONAL BONUS: Check if n samples matches expected value calculated from duration and sample rate
– Duration = 2026-05-14 01:38:49.998019 UTC to 2026-05-14 01:39:49.998015 UTC = 59.999996 sec
– Sample rate = 240 kHz = 240,000 Hz = 240,000 samples/sec
– Expected total samples = 59.999996 sec × 240,000 samples/sec = 14,399,999.04 samples
= OFF by 0.96000000089 samples (aka 0.000004 seconds) from expected

The data payload and checksum of $START lines in DMON2 log files can be decoded as follows…

Example $START line in log file (from UVI2_PRSTT_North_20260428_001.log):

1777471732,6505,$START,69F210F40000 4C232435 00010000*13

Characters 2-9 (n=8) after $START: Date-time in Unix epoch UTC

69F210F4 hexadecimal = **1777471732** decimal (sec Unix epoch) = **2026-04-29 14:08:52** UTC

Characters 10-13 (n=4): Milliseconds after date-time in Unix epoch UTC

0000 hex = **0** decimal (milliseconds) →

2026-04-29 14:08:52 UTC + 0 milliseconds = **2026-04-29 14:08:52.000** UTC

Characters 14-21 (n=8): DMON2 board ID for the specific DMON2 used in the mission

4C232435

Characters 22-29 (n=8): Flash memory size (MB)

00010000 hex = **65536** decimal (MB) = **64** GB (binary units) of flash memory available on the DMON2

**from Baumgartner "The DMON2 application firmware saves data (e.g., audio, logs, configuration information) to flash memory in chunks as they are generated in real time, and d3host assembles those chunks into .dtg files that are then unpacked into .wav, .log, .xml and .err files by the d3read program"*

Characters 31-32 (n=2): Checksum indicating if any errors exist in the $START line

13 **see last page of this tab for details on calculating the checksum for each line*

The data payload and checksum of $END lines in DMON2 log files can be decoded as follows…

Example $END line in log file (from UVI2_PRSTT_North_20260428_098.log):

1779941782,5760,$END,6A17C1960006 4C2324352*2D

Characters 2-9 (n=8) after $END: Date-time in Unix epoch UTC

6A17C196 hexadecimal = **1779941782** decimal (sec Unix epoch) = **2026-05-28 04:16:22** UTC

Characters 10-13 (n=4): Milliseconds after date-time in Unix epoch UTC

0006 hex = **6** decimal (milliseconds) = **0.006** seconds →

2026-05-28 04:16:22 UTC + 6 milliseconds (0.006 sec) = **2026-05-28 04:16:22.006** UTC

Characters 14-21 (n=8): DMON2 board ID for the specific DMON2 used in the mission

4C232435

Character 22 (n=1): Shutdown code indicating DMON2 is properly shutting down and entering hibernation mode

2

Characters 24-25 (n=2): Checksum indicating if any errors exist in the $END line

2D **see last page of this tab for details on calculating the checksum for each line*

How to calculate expected checksum values of DMON2 log file lines

- All characters between "$" and "*" in line get checked
- Calculate binary "running result" value by stacking binary values of last row and current row
  - **different bits between rows = 1 and same bits = 0**

0 1 0 1 0 0 1 1 ← binary running result from all previous characters (row #1, col 5 below)
0 1 0 1 0 1 0 0 ← binary of current character derived from ASCII decimal (row #2, col 4 below)
0 0 0 0 0 1 1 1 ← output binary running result for current row (row #2, col 5 below)

Example checksum calculation for line:
`1777471732,6505,$START,69F210F400004C23243500010000*13`

#	Char	ASCII decimal	Binary	Binary running result	= Decimal	= Hexadecimal result final row
	start			*00000000*	*0*	
1	S	83	01010011	01010011	83	
2	T	84	01010100	00000111	7	
3	A	65	01000001	01000110	70	
4	R	82	01010010	00010100	20	
5	T	84	01010100	01000000	64	
6	,	44	00101100	01101100	108	
7	6	54	00110110	01011010	90	
8	9	57	00111001	01100011	99	
9	F	70	01000110	00100101	37	
10	2	50	00110010	00010111	23	
11	1	49	00110001	00100110	38	
12	0	48	00110000	00010110	22	
13	F	70	01000110	01010000	80	
14	4	52	00110100	01100100	100	
15	0	48	00110000	01010100	84	
16	0	48	00110000	01100100	100	
17	0	48	00110000	01010100	84	
18	0	48	00110000	01100100	100	
19	4	52	00110100	01010000	80	
20	C	67	01000011	00010011	19	
21	2	50	00110010	00100001	33	
22	3	51	00110011	00010010	18	
23	2	50	00110010	00100000	32	
24	4	52	00110100	00010100	20	
25	3	51	00110011	00100111	39	
26	5	53	00110101	00010010	18	
27	0	48	00110000	00100010	34	
28	0	48	00110000	00010010	18	
29	0	48	00110000	00100010	34	
30	1	49	00110001	00010011	19	
31	0	48	00110000	00100011	35	
32	0	48	00110000	00010011	19	
33	0	48	00110000	00100011	35	
34	0	48	00110000	00010011	19	**13 (checksum for this $START line!)**